

Parallel Decoder Transformer: Planner-Conditioned Latent Coordination for Model-Intrinsic Parallel Generation

Logan Robbins*

July 2026

Abstract

Autoregressive language models expose one causal token frontier, even when a requested document contains several sections that could be developed concurrently. Existing parallel-generation systems commonly create external branches around an otherwise unchanged model. We instead formulate *model-intrinsic parallel generation*: a single trained architecture owns multiple causal frontiers and produces one next-token distribution for each frontier in every synchronized decoding round. The proposed Parallel Decoder Transformer (PDT) retains a frozen shared lower knowledge trunk and replaces the upper trunk with three independently parameterized physical decoder stacks. A prompt-time set planner produces three unordered continuous outlines. Each outline is hard-routed to one decoder as persistent Plan-KV memory, while a finite product-quantized notes bus carries block-delayed latent messages among the decoders. The resulting factorization preserves autoregression within each lane but makes same-round lane tokens conditionally independent given the source, their plans, private histories, and previously committed messages.

We describe a source-grounded training methodology for long-form historical exposition. Cited atomic facts receive exactly one owner lane, optional reference lanes, hard negatives, exact outline assignments, and token-aligned cross-lane dependencies. The objective combines lane-local next-token likelihood with unordered plan matching, fact routing, outline progress, owner/reference/absent write supervision, presentation order, and finite-channel regularization. A staged curriculum separately tests the oracle-plan executor, the planner, and their joint operation. We also register causal evaluations based on plan swaps, plan removal, delayed-message ablation, a parameter-matched self-only control, exact dependency-token likelihood, and lane-specific human fact audits. The architecture and evaluation pipeline are implemented; scientific training and held-out evaluation are in progress. Accordingly, this paper presents the theory, design, and falsifiable protocol, not a positive empirical result.

1 Introduction

A conventional decoder-only Transformer represents a distribution over one sequence, $p(y \mid x) = \prod_t p(y_t \mid x, y_{<t})$, and therefore exposes one next-token decision at a time. This factorization is appropriate when the next passage logically depends on all preceding prose. It is unnecessarily restrictive when a response can be decomposed into complementary sections whose local prose is sequential but whose development is partly concurrent. Long-form histories, operating procedures, technical briefings, and other structured documents often have this form.

*Independent researcher. ljrweb@gmail.com

Prior work demonstrates several ways to exploit such structure. Skeleton-of-Thought first elicits a textual skeleton and expands its points through parallel calls or batched branches [Ning et al., 2024]. APAR trains a model to emit fork control structure and changes the attention topology of the resulting paragraph tree [Liu et al., 2024]. PASTA learns an annotation language with asynchronous promises and explicit synchronization [Jin et al., 2025]. These systems establish that generation need not always follow the final reading order. They do not answer the architectural question studied here: can the model itself contain several persistent, physically distinct causal frontiers, learn which part each frontier should write, and exchange only a bounded latent summary of completed work?

We call this setting *model-intrinsic parallel generation*. “Intrinsic” does not mean that all computation is free or that concurrency guarantees a wall-clock speedup. It means that planning, semantic assignment, frontier state, and cross-frontier communication are learned components of one model rather than a controller issuing independent language-model requests. The model returns three next-token distributions in one grouped forward operation. Each frontier owns private upper-layer parameters and KV state, so decoder identity is a physical axis rather than a natural-language role label.

The distinction matters for both open-domain research and closed-loop deployment. Retrieval-augmented generation adds an external non-parametric memory at inference time [Lewis et al., 2020]. That is appropriate when connectivity and an index are available. Our motivating deployment class also includes disconnected or fixed-corpus systems—for example, an onboard model trained around corporate procedures, maintenance manuals, or emergency protocols. In that setting, the source packet and pretrained trunk provide the available knowledge, and the question is whether the model can internally partition and coordinate a long answer without an online retrieval service. The historical domain used in this paper is a controlled testbed for that question, not the limit of the proposed architecture.

The design makes four contributions.

1. We give a causal factorization for three synchronized, lane-local autoregressive sequences. Same-round tokens never observe one another; cross-lane influence is permitted only through messages committed at an earlier block boundary.
2. We introduce a concrete architecture with a frozen shared lower knowledge trunk, a continuous unordered planner, and three independently parameterized upper decoder stacks. Persistent Plan-KV makes assignment structural and inspectable; private caches prevent one lane from acquiring another lane’s token history.
3. We define a finite delayed communication channel. Every completed 32-token lane block may publish four 8-bit product-VQ indices. Producer and lag headers address the message, while the learned code tuple is the entire transmitted payload.
4. We specify source-grounded data, losses, curriculum, controls, and causal gates that distinguish four questions: whether the physical executor can follow oracle plans, whether the planner learns decomposition, whether plan assignment causes fact ownership, and whether sibling messages help exactly where delayed dependencies require them.

The central claim is therefore architectural and falsifiable. The design creates three genuine causal frontiers and a lawful path for coordination. It does not follow that the planner will learn useful decompositions, that the notes channel will be used, or that the resulting prose will be better than a serial baseline. Those are the experiments now in progress.

2 Problem Formulation

2.1 Three sections, not three guesses

Let x contain a source document and an expository request. The required output is a tuple of exactly three multi-paragraph sections $Y = (y^1, y^2, y^3)$ and a presentation permutation $\pi \in S_3$. The reader sees $y^{\pi_1} \| y^{\pi_2} \| y^{\pi_3}$. There is no fourth synthesis decoder and no serial rewriting pass. The three sections are complementary parts of one answer, not independent samples from which a best response is selected.

The planner maps x to three semantic plans $Z = \{z^1, z^2, z^3\}$, where each $z^k = (z_1^k, \dots, z_N^k)$ is an ordered outline of at most N nodes. The lane-level collection is an unordered set: a physical decoder does not have a permanent role such as “background,” “event,” or “consequences.” Presentation order is predicted separately from physical address.

Each source fact f has exactly one owner lane $o(f)$. A different lane may be licensed to reference it, but a fact written only by the wrong lane does not satisfy ownership. This requirement rules out the weak objective in which the union of three outputs receives credit regardless of who wrote what.

2.2 Block-delayed causal factorization

Generation is divided into blocks of τ lane-local tokens; the canonical implementation uses $\tau = 32$. Let $y_{b,r}^k$ be token r of block b in lane k , and let $y_{<b,r}^k$ denote all earlier tokens in that lane. Let $\mathcal{M}_{<b}$ be the set of dynamic messages whose publication time plus delivery delay is no later than block b . With a one-block delay, a message written after block $b - 1$ can first affect block b .

The model uses the factorization

$$p_{\Theta}(Y, Z, \pi \mid x) = p_{\phi}(Z, \pi \mid x) \prod_b \prod_{k=1}^3 \prod_{r=1}^{|y_b^k|} p_{\Theta,k}(y_{b,r}^k \mid x, z^k, y_{<b,r}^k, \mathcal{M}_{<b}). \quad (1)$$

For a fixed synchronized microstep (b, r) , the three current tokens are conditionally independent given the state on the right side of Equation 1. This is not an assertion that the three sections are globally independent. Their dependence is mediated by the common source, the jointly produced plan set, and previously committed messages. Same-round communication is prohibited because it would introduce an ordering among lanes and destroy the claimed concurrency.

2.3 Structural requirements

Table 1 separates the required mechanism from weaker implementations that do not test the same hypothesis.

Requirement	Consequence
Three physical upper stacks	The upper Transformer weights contain an explicit three-way decoder axis. Three batch rows through one parameter set are a control, not the proposed architecture.
Private causal state	Every lane has private upper-layer KV state. No token from another lane enters that cache.
Hard plan address	Lane k reads only its assigned outline nodes through Plan-KV cross-attention. The assignment is not inferred from a textual “D1” tag.
Synchronous advance	One grouped call returns one next-token distribution for every active lane. Serial Python calls do not satisfy the concurrency claim.
Delayed atomic communication	All lane writes at a boundary are committed before any next-block read. No within-round lane order exists.
Exchangeable semantics	All six semantic-plan-to-physical-decoder bindings occur in training, so physical identity cannot become a permanent content role.
No serial synthesizer	The visible document consists only of the three generated sections arranged by the predicted presentation order.

Table 1: Mechanism requirements for model-intrinsic parallel generation.

3 Parallel Decoder Transformer

3.1 Overview

Figure 1 shows the complete topology, while Figure 2 expands one physical upper layer. A revision-pinned decoder-only Transformer is split at layer L_f . Layers below L_f form a single frozen parameter trunk. Layers at and above L_f are copied into three independently trainable physical stacks. A prompt-time planner reads frozen prompt states and produces three continuous outlines. Each upper layer receives its lane’s persistent outline and the currently visible dynamic-note window through separate cross-attention residuals.

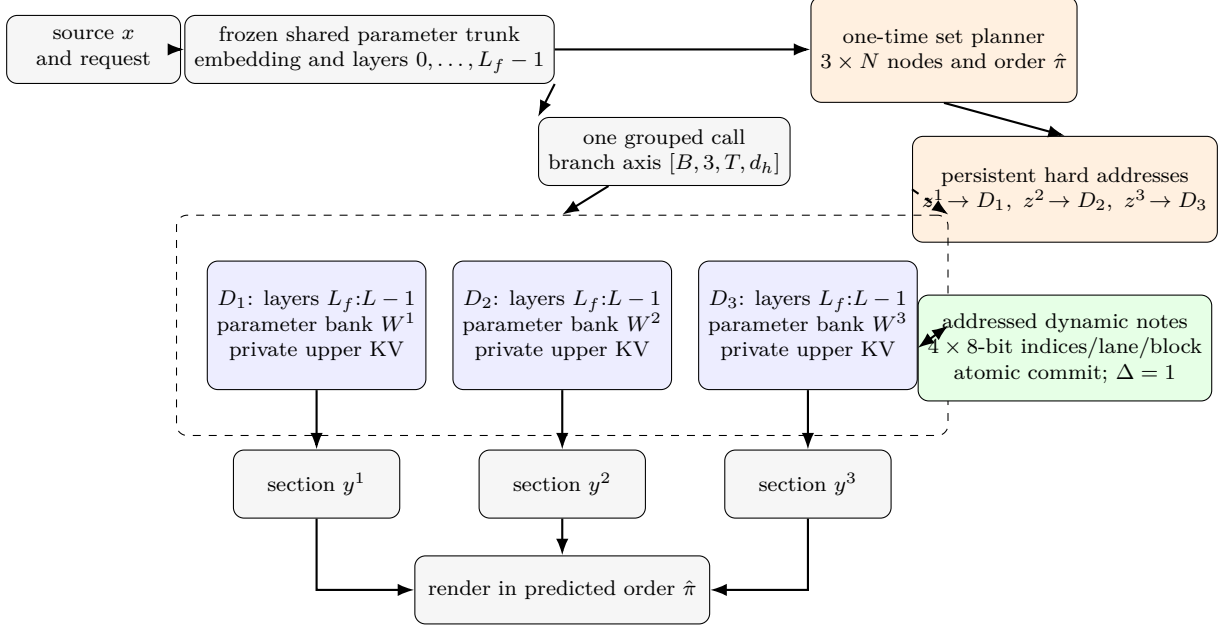


Figure 1: System topology. Solid arrows carry token states or planner computation; dashed arrows carry persistent plan memory or block-delayed latent messages. The lower parameter trunk is shared, but post-prompt lane states are distinct. The three upper stacks advance in one grouped operation and share neither base weights nor upper-layer KV caches.

3.2 Frozen shared knowledge trunk

Let $T_{\theta_{lo}}$ be the embedding layer and lower L_f Transformer layers. Its parameters are frozen. For an identical source prompt, prefill below the fork is evaluated once and the resulting cache is expanded onto the three lane rows. This removes redundant prompt computation.

Parameter sharing must not be confused with a shared hidden trajectory. After the lanes emit different tokens, those token rows pass through the same lower weights in one grouped batch but retain separate lower-layer cache rows and separate hidden states. The shared trunk is therefore a common knowledge function applied to three causal histories, not a single state copied forever.

3.3 Continuous set planner

The planner runs once before generation. The normalized prompt states $H^x = T_{\theta_{lo}}(x)$ are projected from the trunk width d_h to planner width d_p . Three groups of N learned queries cross-attend to H^x through a small Transformer decoder:

$$\hat{Z}, \hat{V}, \hat{s} = P_\phi(H^x), \quad \hat{Z} \in \mathbb{R}^{3 \times N \times d_p}. \quad (2)$$

$\hat{V}$ contains node-validity logits and $\hat{s} \in \mathbb{R}^3$ contains presentation scores. The canonical instance uses $N = 8$, $d_p = 512$, two planner layers, and eight attention heads.

The planner’s three lane groups are trained as a set. The loss enumerates all six lane permutations and differentiates only through the minimum-cost assignment. This follows the general principle that set-valued targets should not acquire meaning from arbitrary storage order [Lee et al., 2019]. Within a selected lane, however, outline-node order is meaningful.

The planner never consumes generated tokens and is not recurrently called. This prevents a hidden serial planner loop from reintroducing one-step-at-a-time coordination.

3.4 Three physical decoder stacks

Let the pretrained upper layers be $T_{\theta_{hi}}^{L_f:L-1}$. We clone them into $D_{\theta_1}, D_{\theta_2}, D_{\theta_3}$. Every self-attention projection, MLP projection, and RMS normalization weight has a leading physical-decoder axis:

$$W \in \mathbb{R}^{3 \times d_{out} \times d_{in}}. \quad (3)$$

The three slices are initialized identically but are distinct parameters with independent gradients. Grouped matrix multiplications advance the explicit decoder axis in one operation. Each upper attention layer also stores private keys and values with shape $[B, 3, H_{kv}, T, d_{head}]$.

This design is materially different from three prompt labels passed through a shared decoder. A lane cannot recover its assignment from a learned “meta-key” inside a shared cache because no such inference is required: the private cache axis, branch parameter slice, and hard-routed plan memory already determine the address. The shared coordination modules process all rows with common parameters, but the Qwen self-attention and MLP path above the fork is independent for each physical lane.

3.5 Persistent hard-routed Plan-KV

Planner nodes are projected from d_p to a 256-wide memory representation. At every physical upper layer ℓ , lane k reads only its own valid nodes:

$$\tilde{z}_n^k = W_z z_n^k + e_n^{\text{node}}, \quad (4)$$

$$A_{\text{plan}}^\ell(h^k, z^k) = W_O^\ell \text{softmax} \left(\frac{(W_Q^\ell h^k)(W_K^\ell \tilde{z}^k)^\top}{\sqrt{d_a}} \right) W_V^\ell \tilde{z}^k. \quad (5)$$

The result enters through a learned gated residual. Individual nodes remain available as keys and values; mean pooling or a single additive plan vector is not the canonical path. Plan memory is read-only and persists for the complete generation.

3.6 Finite delayed dynamic notes

Static plans define intended work. Dynamic notes summarize work actually completed. At the end of block b , lane k ’s block-final hidden state is projected to $u_b^k \in \mathbb{R}^{256}$, split into $M = 4$ subvectors, and quantized:

$$c_{b,m}^k = \arg \min_{c \in \{1, \dots, C\}} \|u_{b,m}^k - e_{m,c}\|_2^2, \quad M = 4, \quad C = 256. \quad (6)$$

The transmitted payload is the tuple $(c_{b,1}^k, \dots, c_{b,4}^k)$, whose capacity is

$$M \log_2 C = 4 \log_2 256 = 32 \text{ bits per lane block}. \quad (7)$$

The receiving process decodes those indices through the learned codebooks. The reconstructed 256-dimensional tensor is a local representation, not a 256-value message placed on the logical channel [van den Oord et al., 2017].

Every note includes non-payload headers identifying its producer and age. Shared Notes Cross-Attention (SNC) adds learned producer, message-kind, and log-lag embeddings before forming keys and values. A receiver’s hidden state forms the query, so the same addressed note can be used differently by each lane. The window retains up to 16 blocks of history.

Writes are atomic at a block boundary. A note published at block b is invisible during block b and becomes readable at block $b+1$. All three writes are committed before any receiver begins the next

block. Both Plan-KV and SNC use zero-initialized output projections and closed residual gates, so the untrained extension begins as the cloned pretrained decoder rather than an uncontrolled perturbation.

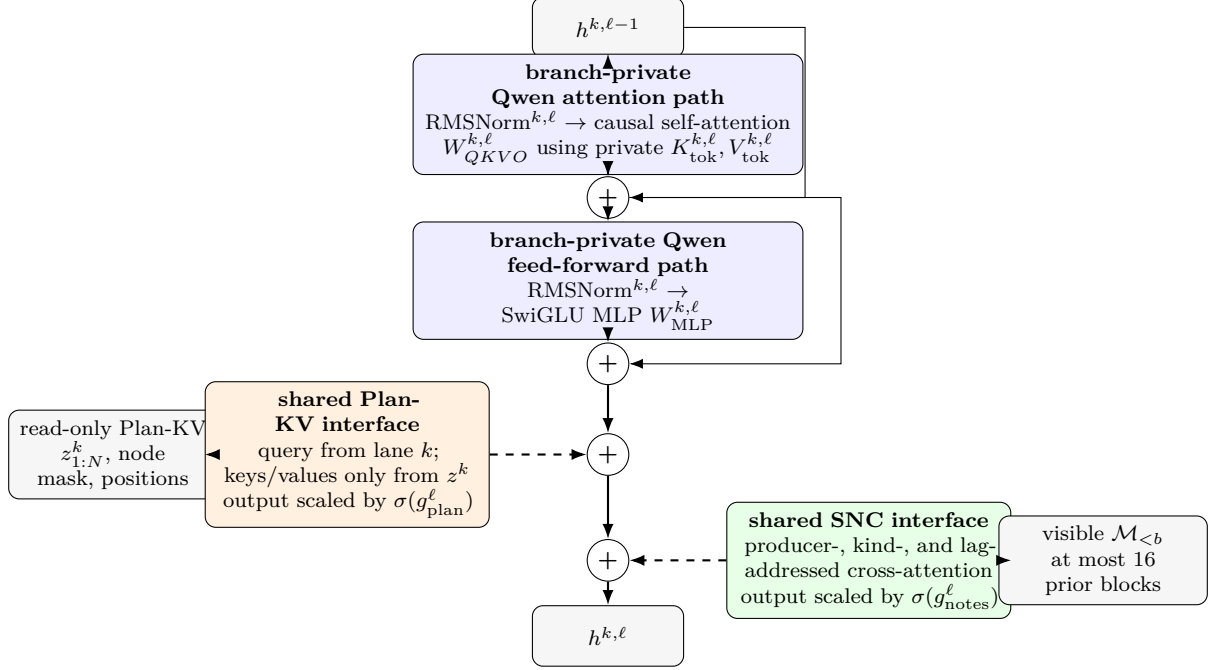


Figure 2: Anatomy of upper layer ℓ for physical lane k . The Qwen self-attention, normalization, MLP weights, and token KV cache are independently parameterized for each lane. Plan-KV and SNC have shared interface parameterizations, but operate on lane-private token states and structurally addressed memories. This sequence is repeated at every physical upper layer.

4 Synchronized Parallel Generation

Natural generation follows one canonical loop.

1. Encode the source prompt and run the planner once. Resolve three valid plan memories and one presentation order.
2. Prefill the common prompt below the fork once, expand the lower cache to three rows, and pass all rows through their physical upper stacks with the assigned Plan-KV.
3. At round t , assemble one token row per physical lane and execute one grouped lower-plus-upper forward. Sample one token from each active lane’s returned distribution.
4. When every active lane reaches a 32-token boundary, compute all product-VQ writes and commit them together. Build the next block’s addressed note windows only after the commit.
5. A lane that emits EOS becomes logically inactive but remains a physically present masked row in the packed cache. Other lanes continue without falling back to separate model calls.
6. Decode each lane independently and concatenate the three sections in predicted presentation order.

Figure 3 makes the synchronization and message delay explicit. A vertical slice through the three lane rows is one grouped decoding round. No lane can observe a sibling’s token or block-end code from the current block.

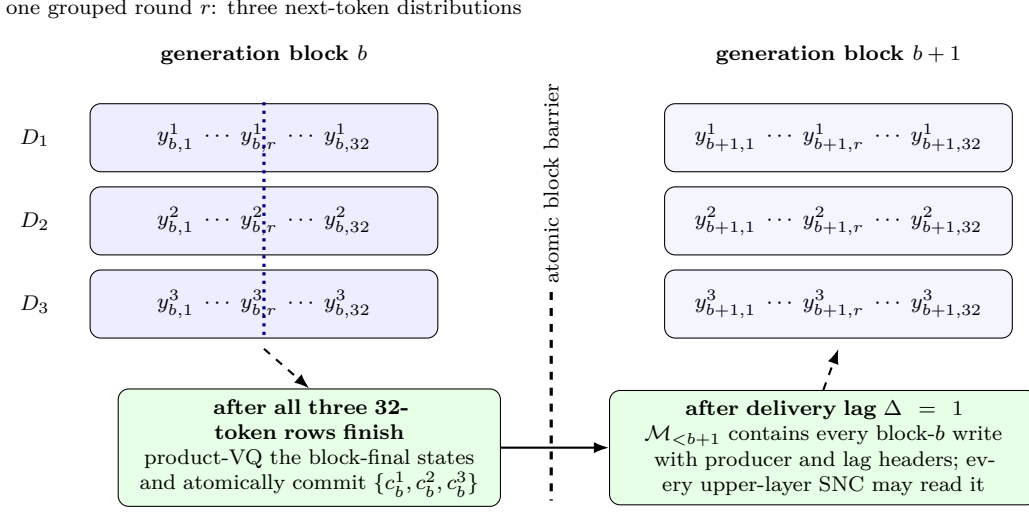


Figure 3: Synchronized causal timeline for block size $\tau = 32$. Each microstep advances all active physical lanes in one grouped call. After the final microstep, the three product-VQ writes are computed from the completed lane states and committed as one atomic set. They cannot affect block b and first enter the addressed SNC windows for block $b + 1$.

This algorithm emits up to three sampled tokens per synchronization round while preserving token-level autoregression inside every section. If the final lane lengths are L_1, L_2, L_3 , a serial concatenated decoder requires $R_{\text{serial}} = \sum_k L_k$ decode rounds. The parallel frontier has algorithmic span $R_{\text{PDT}} = \max_k L_k$ after planning and prefill. Its maximum span reduction is therefore

$$\rho_{\text{span}} = \frac{\sum_{k=1}^3 L_k}{\max_k L_k} \leq 3. \quad (8)$$

Equation 8 is a dependency-depth statement, not a hardware speed claim. PDT performs more parameterized work than a one-stack decoder because the upper weights are triplicated, and real latency depends on kernels, memory traffic, accelerator occupancy, cache length, and lane balance. Hardware changes can alter that mapping from span to time without changing the causal potential of the architecture. We therefore report synchronization depth, arithmetic work, memory footprint, and measured wall time separately.

5 Source-Grounded Long-Form Supervision

5.1 Why historical exposition

Short answers and question-answer pairs do not test the proposed mechanism. A lane can satisfy them before persistent plans, ownership, and delayed coordination become meaningful. The first study instead uses historical events and processes whose source material naturally supports several long, complementary sections. Every teacher section must contain 700–1,000 Qwen tokens and four to twelve connected paragraphs. Lists, timelines, biographies as primary topics, and catalog-like pages are excluded.

Historical text also supplies checkable temporal constraints. A natural decomposition should not require a reader to understand an undefined event before its causes or actors are introduced. This is what “natural” means in the data contract: not that a record proves latent coordination, but that its presentation order and cross-section references obey the document’s causal and expository prerequisites.

5.2 Immutable Wikimedia source ingress

Each source begins from one exact English Wikimedia revision. The immutable raw bundle retains wikitext, rendered semantic HTML, PageAssessments metadata, reference records, revision identifiers, licenses, and cryptographic digests. Only a derived representation is model-visible.

The renderer is a strict DOM allowlist. It keeps the article title, hierarchical content headings, and ordinary prose paragraphs. Inline links contribute only their display text. It excludes tables, lists, infoboxes, templates, figures, captions, galleries, maps, citation markers, footnotes, bibliographies, URLs, navigation, categories, authority-control blocks, hatnotes, pronunciation, mathematics, code, and edit artifacts. Reference identities remain audit metadata and are never concatenated into the student prompt.

Articles are accepted whole; no oversized page is prefix-truncated and no fragments are stitched. Table 2 lists the preregistered filter.

Gate	Requirement
Topic and age	English main-namespace event or process; non-redirect, non-disambiguation; event ended at least 25 years before acquisition.
Assessment	Relevant history-oriented WikiProject assessment of FA, GA, A, or B; no citation, neutrality, accuracy, original-research, cleanup, or hoax maintenance template.
Complete cleaned body	3,000–7,000 pinned-Qwen tokens, at least six substantive sections, and at least twelve substantive paragraphs.
Citations	At least 30 inline reference occurrences, 15 distinct cited works, eight scholarly or institutional sources, and citations on at least 70% of substantive paragraphs.
Source concentration	No cited work contributes more than 25% of all reference occurrences.
Split integrity	Related pages and manually reviewed family keys are assigned to one train, validation, or test split before teacher generation.

Table 2: Whole-article source gates for the historical corpus.

5.3 Two-stage teacher construction

Teacher data are produced in two schema-constrained stages.

Stage A: cited facts. For each accepted source, the teacher extracts 18–48 atomic facts. Every fact contains an exact source quote, character offsets, paragraph ID, and local reference IDs that must agree with the immutable source record. The inventory must span at least twelve source paragraphs and four source sections. Each fact also receives a plausible hard negative that changes an entity, date, quantity, or relation.

Stage B: joint three-lane answer. One request receives the complete source and validated fact inventory and returns one expository prompt, exactly three plans, all three target sections, and their

presentation order. The teacher generates the lanes jointly rather than through three independent calls, because cross-lane ownership and references are properties of the complete document.

Every plan has four to eight outline nodes. A node specifies its expository objective, owned facts, permitted reference facts, dependencies, and target paragraphs. Every source fact has one and only one OWNER lane; each other lane labels it REFERENCE or ABSENT. An OWNER or REFERENCE label includes an exact quote in the target section. Hard negatives must remain absent everywhere.

Cross-lane references are optional. When present, a dependency names the owner plan, owner node, source quote, receiver node, receiver quote, and fact IDs. The owner paragraph must precede the receiver in presentation order, and the complete source quote must fall in a generation block visible at least one block before the first receiver token. Records with zero dependencies are valid; the data compiler does not invent communication to make the notes bus look useful.

5.4 Tokenized training record

Retokenization uses the exact student tokenizer and appends one EOS token to each target. A frozen 1,024-dimensional BGE encoder [Xiao et al., 2023] embeds every true fact and hard negative. The same encoder embeds a textual representation of every outline node; a fixed Gaussian Johnson–Lindenstrauss projection maps those targets to 512 dimensions. Retokenization materializes:

- true and hard-negative fact embeddings;
- eight padded continuous plan targets per lane and node-validity masks;
- exact owned-fact-to-outline-node routing;
- one active outline node per 32-token target block;
- OWNER/REFERENCE/ABSENT labels at the exact lane and block where evidence occurs;
- exact dependency-token masks and source-block-to-target-token edges;
- the reader-facing presentation ranks.

The first pilot targets 128 accepted records balanced across eight historical categories. Every record is manually inspected. A 2,048-record corpus is the architecture gate; a larger corpus is not authorized until the oracle executor and learned planner show value. Four separately curated source-grounded examples currently exercise the complete schema and trainer interface. They are data-contract inspection records, not a training corpus or an empirical result.

6 Learning Objective

6.1 Unordered plan matching

For example i , let $\hat{z}_{k,n}$ and $\hat{v}_{k,n}$ be predicted plan nodes and validity logits, and let $z_{k,n}$ and $v_{k,n}$ be teacher targets. For each lane permutation $\sigma \in S_3$, define

$$C_i(\sigma) = \frac{\sum_{k,n} v_{k,n} [1 - \cos(\hat{z}_{\sigma(k),n}, z_{k,n})]}{\sum_{k,n} v_{k,n}} + \frac{1}{3N} \sum_{k,n} \text{BCE}(\hat{v}_{\sigma(k),n}, v_{k,n}) . \quad (9)$$

The selected assignment is $\sigma_i^* = \arg \min_{\sigma \in S_3} C_i(\sigma)$. The discrete choice is detached; gradients flow through the selected predicted nodes. All downstream semantic axes are then expressed in that matched order.

6.2 Execution and semantic losses

Lane token likelihood. $\mathcal{L}_{\text{tok}}$ is exact teacher-forced next-token cross entropy over every valid token, including each lane’s EOS. The first token in every target block is scored from the preceding prompt or block-final distribution, so block packing does not drop boundary likelihood.

Owned-fact routing. The route head scores every plan node against all true and hard-negative fact embeddings. $\mathcal{L}_{\text{route}}$ gives equal aggregate weight to positive owned node–fact pairs and negative pairs. Reference facts are not ownership targets for this head.

Outline progress. The block-final hidden state is compared with the assigned plan nodes. $\mathcal{L}_{\text{progress}}$ is cross entropy over the node whose target paragraph occupies most of that block.

Fact writing. For every lane, target block, and fact query, a classifier predicts OWNER, REFERENCE, or ABSENT. $\mathcal{L}_{\text{write}}$ uses inverse-frequency class weights $w_c = n/(3n_c)$, preventing the many absent labels from defining a trivial optimum. A fact in the wrong physical lane is explicitly penalized even if it appears somewhere in the combined answer.

Dynamic-note alignment. For every active block write, the quantized note is paired with its pre-quantized projection. $\mathcal{L}_{\text{note}}$ is symmetric InfoNCE over all writes in the packed document with temperature 0.07. Other lane/block writes supply negatives.

Presentation order. Given matched lane scores s and teacher order $\pi = (\pi_1, \pi_2, \pi_3)$, a Plackett–Luce loss is

$$\mathcal{L}_{\text{order}} = -\log \frac{e^{s_{\pi_1}}}{\sum_j e^{s_j}} - \log \frac{e^{s_{\pi_2}}}{e^{s_{\pi_2}} + e^{s_{\pi_3}}}. \quad (10)$$

Finite-channel regularization. For continuous subvector u and selected code e ,

$$\mathcal{L}_{\text{commit}} = \|u - \text{sg}[e]\|_2^2, \quad (11)$$

$$\mathcal{L}_{\text{codebook}} = \|\text{sg}[u] - e\|_2^2. \quad (12)$$

$\mathcal{L}_{\text{usage}}$ is the normalized deficit between maximum code entropy and the empirical marginal entropy in each product codebook. It discourages collapse without increasing channel capacity.

6.3 Combined objective

After plan matching, the canonical full objective is

$$\begin{aligned} \mathcal{L} = & 1.00\mathcal{L}_{\text{tok}} + 1.00\mathcal{L}_{\text{plan}} + 1.00\mathcal{L}_{\text{route}} + 0.50\mathcal{L}_{\text{progress}} \\ & + 1.00\mathcal{L}_{\text{write}} + 0.25\mathcal{L}_{\text{note}} + 0.10\mathcal{L}_{\text{order}} \\ & + 0.25\mathcal{L}_{\text{commit}} + 1.00\mathcal{L}_{\text{codebook}} + 0.10\mathcal{L}_{\text{usage}}. \end{aligned} \quad (13)$$

These coefficients are a registered starting configuration, not an asserted optimum. The architecture screen varies learning rate, weight decay, fork depth, planner capacity, notes width, codebook

geometry, and residual-gate initialization before any selected interactions are tested in a separate registered run.

7 Curriculum and Optimization

Training separates capability questions that a single end-to-end run would confound. Table 3 gives the four stages.

Stage	Steps and plan	Trainable components	Scientific question
0	0–3,749; oracle outline	Three upper decoder banks, Plan-KV, SNC, semantic heads, note writer	Can the physical executor realize a correct three-way decomposition when the plan is given?
1	3,750–9,999; learned outline	Planner only; the executor and semantic heads are frozen	Can the planner reproduce an unordered teacher plan set through a fixed executor interface?
2	10,000–24,999; learned outline	Planner, upper decoder banks, Plan-KV, SNC, semantic heads, note writer	Can planning and execution adapt jointly while the lower knowledge trunk stays fixed?
3	25,000–49,999; learned outline	Same topology as Stage 2	Does continued joint training improve the same mechanism without introducing a second path?

Table 3: Canonical training curriculum. Stage-specific loss weights disable objectives whose required module is frozen.

Every training example randomly maps its three semantic plans and target sections onto the three physical decoders. Across examples, all six permutations are eligible. This is stronger than sharing branch weights: the physical upper stacks may diverge and learn different numerical representations, but none can rely on a fixed semantic job. At evaluation, a plan swap holds prompt and physical weights fixed while moving semantic responsibility; ownership must move with the plan.

The shared lower trunk remains frozen in all stages. Physical branch weights use FP32 optimizer masters and BF16 grouped computation. Packed teacher forcing processes one 32-token block for each lane in a single physical-frontier call, then publishes all active block notes. AdamW uses gradient clipping, cosine decay, and exact stage-aware module manifests. Checkpoints bind the trunk model and revision, fork layer, decoder count, coordination condition, optimizer, scheduler, step, and stage. A checkpoint cannot load into a different architecture or a bus/self-only condition with the same filename.

8 Evaluation Protocol and Current Status

8.1 Questions, interventions, and gates

The evaluation is designed to prevent one success from standing in for another. Table 4 lists the principal questions. All confidence intervals are computed over documents rather than individual facts.

Question	Conditions	Preregistered evidence
Executor capability	Oracle teacher plan	Owner-fact recall lower bound $\geq 90\%$; unauthorized leakage upper bound $\leq 10\%$; outline completion $\geq 90\%$; every lane remains long-form.
Planner capability	Oracle plan vs. learned plan vs. zero plan	Learned owner recall retains at least 80% of oracle; zeroing all plan nodes drops owner recall by at least 30 percentage points.
Plan causality	Learned plan vs. physical D_1/D_2 plan swap	At moved addresses, swapped outputs retain at least 80% of learned-plan owner recall; moved-address scoring exceeds deliberately unmoved scoring by at least 30 percentage points.
Dynamic communication	Normal bus vs. dynamic-note removal; exact dependency and nondependency tokens	Report document-bootstrap change in NLL at both token sets and their difference. Plan memory remains intact in both conditions.
Sibling information	Bus vs. parameter-matched self-only memory	Compare owner/reference realization, delayed-reference timing, duplication, and prose quality under identical plan paths and training budgets.
Parallel execution	One lane, three serial lanes, packed three-lane frontier	Report decode rounds, complete-answer latency, aggregate tokens/s, prefill, decode time, peak memory, and per-round call/cache traces.

Table 4: Evaluation questions and preregistered evidence. Thresholds are gates for future measurements, not current results.

8.2 Fact ownership and long-form quality

Free generation stops each lane at its own EOS or at 1,000 new tokens. A valid lane contains at least 700 tokens and four paragraphs, with no more than 10% of sentences shorter than eight words. These checks reject the short-sentence failure mode directly rather than treating token count alone as long-form quality.

Fact presence is not measured by exact string identity. A pinned NLI cross-encoder scores each atomic claim against sentence-level evidence units in each physical lane. Its threshold is calibrated once on disjoint teacher prose containing true and hard-negative claims, then frozen for held-out generation. This automatic score is a screen, not final factual evidence. Every condition-lane-fact decision is exported to a blinded queue. Two independent annotators judge the complete queue; a distinct third adjudicator resolves all disagreements and uncertain cases. Final owner recall, reference recall, unauthorized leakage, and hard-negative rate use those adjudicated judgments.

8.3 Causal communication measurement

Teacher-forced communication evaluation reruns the exact target with dynamic notes removed while leaving persistent Plan-KV unchanged. Retokenization has already marked every token overlapping a required receiver quote and proved that its source block is visible under the one-block delay. For token q , define

$$\Delta_q = \text{NLL}_{\text{no-dynamic}}(q) - \text{NLL}_{\text{bus}}(q). \quad (14)$$

The evaluator reports the document-level mean Δ on dependency tokens, the mean on all other active tokens, and their selectivity difference. A global likelihood change is insufficient: the intended mechanism predicts a larger benefit at annotated delayed dependencies.

Additional destructive controls mutate one producer’s finite code, exchange note sources, scramble note directions at matched norm, or close the note gate. These interventions diagnose address and payload sensitivity. They do not replace the primary bus-versus-self-only comparison.

8.4 Scale and architecture controls

The first model rung is a dense Qwen3-4B checkpoint [Qwen Team, 2025]. If its oracle-plan executor fails, the response is not immediately to declare the architecture impossible. Data alignment, ownership labels, routing heads, optimization, cache behavior, and plan delivery are audited first. Only if those checks pass is the same architecture repeated with a dense 14B trunk as a capacity control. The larger model does not change the scientific condition; it asks whether a negative 4B result is a trunk-capacity result.

8.5 Status

At the time of writing, the three physical decoder stacks, private cache, planner, Plan-KV, product-VQ notes bus, packed training rollout, counterfactual runtime, historical source filter, strict teacher schemas, and evaluators are implemented. Four manually audited long-form records exercise the data and trainer contracts, including sparse and zero-dependency cases. These checks establish executable plumbing only. Training on an admissible corpus and held-out causal evaluation are in progress; no quality, coordination, or speedup result is reported in this manuscript.

9 Related Work

Prompted and learned semantic branching. Skeleton-of-Thought externalizes a textual skeleton and expands its points through parallel requests or batched decoding [Ning et al., 2024]. APAR post-trains a model on hierarchical paragraph structure, uses fork control tokens, and restricts attention to ancestors and a branch’s own history [Liu et al., 2024]. PASTA learns a promise-and-sync annotation language so semantically independent chunks can be decoded asynchronously and later rejoined [Jin et al., 2025]. PDT shares the premise that final reading order need not equal generation order. It differs by making three persistent upper decoder stacks part of the model and by conditioning each on a continuous plan plus a finite delayed latent channel.

Concurrent workers with high-bandwidth visibility. Hogwild! Inference lets concurrent model instances attend through a concurrently updated shared cache [Rodionov et al., 2025]. Group Think gives multiple concurrent reasoning trajectories token-level visibility into one another [Hsu et al., 2025]. These approaches explore rich and immediate cross-worker information. PDT studies the opposite communication regime: private causal caches, an explicit one-block barrier, and 32 learned payload bits per lane block. The restriction makes causal attribution and channel capacity directly measurable.

Non-autoregressive and multi-token generation. Non-autoregressive translation predicts many target positions in parallel, trading away the standard left-to-right factorization and often relying on distillation [Gu et al., 2018, Kim and Rush, 2016]. Speculative decoding accelerates sampling while preserving a target model’s output distribution [Leviathan et al., 2023]; Medusa adds multiple decoding heads [Cai et al., 2024], and EAGLE speculates in feature space [Li et al., 2024]. Multi-token prediction trains independent future-token heads above a shared trunk [Gloeckle et al., 2024]. PDT has a different target: each output head owns a complete, persistent autoregressive

section, and the three sections intentionally differ from the sequence a single left-to-right decoder would have sampled.

Set prediction, discrete latents, and external memory. Attention over learned seed queries provides a natural mechanism for set-valued outputs [Lee et al., 2019]; PDT combines learned planner queries with exact three-way assignment. Product vector quantization supplies a differentiable finite alphabet [van den Oord et al., 2017]. Retrieval-augmented generation couples a parametric generator to external non-parametric memory [Lewis et al., 2020]; PDT’s plan and notes are instead internal, document-specific state derived from the current source and generated blocks.

10 Discussion

10.1 Value of physical decoders without constant communication

The dynamic notes bus is not required to fire on every example for the physical split to be meaningful. Three independent upper stacks can adapt their attention and MLP parameters to concurrent long-form trajectories, maintain private causal summaries, and follow different persistent plans without forcing all three roles through one upper parameterization. The output is still three tokens per synchronized round even when the optimal dynamic message is uninformative.

This does not imply that the stacks should become fixed experts. Random semantic-to-physical assignment makes permanent roles actively unreliable. The intended distinction is between *trajectory capacity* and *semantic identity*: each physical stack may learn how to sustain one long concurrent trajectory, while its subject matter is determined by the assigned plan. The self-only and no-dynamic controls measure the value of that physical factorization; the bus ablation measures only the incremental value of sibling communication.

10.2 Why persistent attention rather than a latent seed

A single additive vector can be ignored, overwritten, or entangled with lane state. Plan-KV instead exposes each outline node as an addressable memory at every upper layer and every token. This does not assume that natural-language instructions correspond to a linear direction in the residual stream. It asks the model to learn queries into a persistent structured memory. Complete-plan swaps then provide a causal test: if assignment is real, fact ownership must move while physical weights remain fixed.

10.3 Offline and fixed-corpus systems

The architecture is compatible with systems that cannot query an external service during generation. A domain-specific source packet can be provided in context, or a trunk can be trained on a controlled corpus. The planner and notes bus operate entirely on local hidden states. For operating procedures, vehicle manuals, maritime protocols, or other closed environments, this could support concurrent generation of complementary sections while retaining a bounded and inspectable internal communication path.

This is a deployment hypothesis, not evidence that Wikipedia-trained historical decomposition transfers automatically to safety-critical procedures. Such a transfer would require domain data, expert validation, failure-mode testing, and a separate safety case.

10.4 Falsifiability

The design would be weakened by any of the following outcomes:

- the oracle-plan executor cannot achieve reliable lane ownership, showing that three physical stacks do not by themselves solve assignment;
- the learned planner remains far below the oracle despite sufficient executor performance, showing that decomposition is the bottleneck;
- plan swaps leave facts at their original physical addresses, showing that branches learned fixed roles or ignored Plan-KV;
- dynamic-note removal has no selective effect on annotated dependency tokens, showing that the bus is unused or unnecessary;
- the self-only control matches the bus on dependency-sensitive held-out data, showing that sibling communication adds no measured value;
- grouped execution fails to reduce practical complete-answer latency, showing that algorithmic span is not realized on the tested hardware.

These outcomes are separable. For example, a failed notes bus does not erase the architectural novelty of physical parallel frontiers, but it does falsify the stronger claim that a finite latent channel supports useful cross-lane coordination.

11 Limitations

Three independent upper stacks increase parameters, optimizer state, and upper-layer memory traffic. The canonical 24/12 split is a test point, not proof that half-depth branching is optimal. Realized speed depends on hardware and kernel support, and Equation 8 must not be quoted as measured latency.

The initial evidence domain is English historical exposition. It is selected because facts, chronology, citations, and multi-section structure can be audited, not because history represents every long-form task. Code, mathematics, fiction, and tightly sequential reasoning may admit different decompositions or none at all.

Wikipedia quality assessments and citations are filters, not guarantees that every source sentence is true. Teacher-generated facts and plans can introduce errors even under exact provenance schemas. BGE plan targets, NLI fact screens, and product-VQ codes are model-dependent representations. Human inspection and held-out adjudication remain necessary.

The one-block barrier excludes useful same-round interactions by design. A 32-bit message may be too narrow, while a wider or more frequent channel may erase the intended communication constraint. The architecture screen can measure this trade-off but cannot make it disappear.

Finally, implementation tests show tensor shapes, causal delay, parameter movement, and cache growth. They cannot establish semantic coordination. Until training and held-out evaluation are complete, no result should be inferred from executable plumbing or from untrained generated tokens.

12 Conclusion

The Parallel Decoder Transformer turns long-form parallel generation into a model architecture rather than an external scheduling trick. A frozen shared lower trunk supplies common language and world knowledge; a continuous planner produces an unordered set of three outlines; three independently parameterized upper stacks own private causal frontiers; persistent Plan-KV supplies structural assignment; and a finite delayed notes bus provides a lawful path for cross-lane influence. Under this factorization, the model can emit three tokens per synchronization round without violating autoregression inside any section.

The harder question is whether the architecture learns the intended behavior. The proposed data contract and evaluation separate executor capability, planner decomposition, plan causality, and dynamic communication. Those tests are now in progress. The contribution of this paper is the precise, implementable, and falsifiable design against which the eventual result can be judged.

A Canonical Qwen3-4B Instantiation

The first implementation uses `Qwen/Qwen3-4B-Instruct-2507` at a pinned revision [Qwen Team, 2025]. It has 36 decoder layers and hidden width 2,560. Layers 0–23 form the frozen shared trunk; layers 24–35 initialize each of the three physical upper stacks. Table 5 records the canonical dimensions.

Component	Canonical value
Physical decoder lanes	3
Shared / physical upper layers	24 / 3×12
Planner nodes per lane	8
Planner width / layers / heads	512 / 2 / 8
Plan and notes memory width	256
Plan/SNC attention width / heads	512 / 8
Target block size	32 tokens
Dynamic codebooks / entries	4 / 256
Logical dynamic capacity	32 bits per lane block
Dynamic history	16 blocks
Plan and notes outer gate initialization	−4.0
Shared frozen parameters	2,811,298,304
Physical upper-stack parameters	3,633,509,376
Planner and coordination parameters	84,132,905
Total materialized parameters	6,528,940,585

Table 5: Canonical architecture configuration. Parameter counts are an implementation census, not a performance result.

The physical stack keeps FP32 master parameters and creates BF16 grouped compute views during a recurrent rollout. Frozen lower-layer execution is explicitly outside autograd. Exact causal grouped-query attention is evaluated in bounded query tiles with activation recomputation, and the three branch weights are contracted through grouped matrix multiplication.

B Registered Training Configuration

The canonical optimizer uses AdamW with learning rate 2×10^{-4} , weight decay 0.01, 1,250 warmup steps, cosine decay, gradient-norm clipping at 1.0, batch size one document, and gradient accumulation over 16 documents. The maximum planner prompt is 16,384 tokens; this is a fail-fast ceiling, not a padding length. Source documents are never truncated to meet it.

The architecture screen contains 25 one-factor variants at three independent seeds. It varies model scale, upper-stack depth, planner depth and feed-forward width, notes and attention width, product-code shape, gate initialization, learning rate, weight decay, and bus versus parameter-matched self-only memory. It is compiled before training, and any interaction study is registered after the one-factor screen rather than selected post hoc.

C Reproducibility Commands

The canonical implementation is configured by `configs/pdt_qwen3_4b.yaml`. After the pinned model and data are present, the core commands are:

```
uv sync --frozen
uv run ruff check src scripts tests
uv run mypy src scripts
uv run pytest -q

uv run scripts/compile_architecture_sweep.py \
  --spec configs/pdt_architecture_screen.yaml \
  --output-dir experiments/architecture_screen/configs

uv run scripts/train.py \
  --config configs/pdt_qwen3_4b.yaml \
  --dataset-path data/processed/real_plan/\
qwen3_4b_instruct_2507/train.jsonl \
  --telemetry-dir experiments/qwen3_4b_instruct_2507/real_plan_bus
```

Long-running executions use `nohup`, write verbose logs, and are polled at 15-second intervals. Evaluation uses stage-aware oracle, learned, plan-zero, and lane-swap conditions, then exports every fact decision for blinded human audit.

References

- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2401.10774>.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2404.19737>.

- Jiatao Gu, James Bradbury, Caiming Xiong, Victor O. K. Li, and Richard Socher. Non-autoregressive neural machine translation. In *International Conference on Learning Representations*, 2018. URL <https://arxiv.org/abs/1711.02281>.
- Chan-Jan Hsu, Davide Buffelli, Jamie McGowan, Feng-Ting Liao, Yi-Chang Chen, Sattar Vakili, and Da-shan Shiu. Group think: Multiple concurrent reasoning agents collaborating at token level granularity, 2025. URL <https://arxiv.org/abs/2505.11107>.
- Tian Jin, Ellie Y. Cheng, Zack Ankner, Nikunj Saunshi, Blake M. Elias, Amir Yazdanbakhsh, Jonathan Ragan-Kelley, Suvinay Subramanian, and Michael Carbin. Learning to keep a promise: Scaling language model decoding parallelism with learned asynchronous decoding. In *International Conference on Machine Learning*, 2025. URL <https://arxiv.org/abs/2502.11517>.
- Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Conference on Empirical Methods in Natural Language Processing*, 2016. URL <https://aclanthology.org/D16-1139/>.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International Conference on Machine Learning*, 2019. URL <https://arxiv.org/abs/1810.00825>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2211.17192>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2401.15077>.
- Mingdao Liu, Aohan Zeng, Bowen Wang, Peng Zhang, Jie Tang, and Yuxiao Dong. APAR: LLMs can do auto-parallel auto-regressive decoding, 2024. URL <https://arxiv.org/abs/2401.06761>.
- Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. Skeleton-of-thought: Prompting LLMs for efficient parallel generation. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2307.15337>.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Gleb Rodionov, Roman Garipov, Alina Shutova, George Yakushev, Vage Egiazarian, Anton Sinitsin, Denis Kuznedelev, and Dan Alistarh. Hogwild! inference: Parallel LLM generation via concurrent attention, 2025. URL <https://arxiv.org/abs/2504.06261>.
- Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017. URL <https://arxiv.org/abs/1711.00937>.
- Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general chinese embeddings, 2023. URL <https://arxiv.org/abs/2309.07597>.